

# CTEC 121 Introduction to Programming and Problem Solving

Scarpelli Hall Room 125 · Mondays & Wednesdays, 10:00 AM – 12:20 PM  
September 21 – December 4, 2026 · 5 credits · **\$0 materials**

Bruce Elgort [belgort@clark.edu](mailto:belgort@clark.edu) · (360) 992-2951 · Slack is fastest · [bit.ly/ctec121fall2026](https://bit.ly/ctec121fall2026)

---

## Start here

If you read one part of this document, read this one.

|                     |                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|---------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| WHAT IT COSTS       | <b>\$0.</b> No textbook, no lab fees, no access codes.                                                                                                                                                                                                                                                                                                                                                                                                   |
| WHEN THINGS ARE DUE | <b>Sundays at 11:59 PM Pacific</b> — and most work stays open three more days, through <b>Wednesday at 11:59 PM</b> , at full credit. The exceptions are anything taken in class and the final project.                                                                                                                                                                                                                                                  |
| TIME TO BUDGET      | About <b>10 hours a week</b> . Class is 4 hours 40 minutes of that, so plan on roughly <b>5-6 hours on your own</b> .                                                                                                                                                                                                                                                                                                                                    |
| HOW TO REACH ME     | <b>Slack first.</b> I reply within 24 hours on weekdays.                                                                                                                                                                                                                                                                                                                                                                                                 |
| THE TWO BIG ONES    | The <b>final exam</b> — in class, <b>Monday, November 30</b> . And the <b>final project</b> — due <b>Friday, December 4</b> , a firm date, presented during finals week.                                                                                                                                                                                                                                                                                 |
| THE AI RULE         | The CS50 Duck is always allowed on take-home work. No other AI may write, complete, or suggest code you turn in — unless an assignment says otherwise, and some will. Using one to explain a concept or an error message is fine while you are studying or working on take-home assignments, as long as you say so in a comment. You'll write your code in <b><a href="#">cs50.dev</a></b> , which has the Duck built in and AI autocomplete turned off. |
| IF YOU SLIP UP      | The <b>Regret Clause</b> lets you tell me once, with no penalty. See <i>Academic honesty</i> .                                                                                                                                                                                                                                                                                                                                                           |
| IF YOU FALL BEHIND  | Message me <i>before</i> the deadline. We'll set a new date. This is a real offer, not a trap. If an emergency makes reaching me beforehand impossible, tell me as soon as you can.                                                                                                                                                                                                                                                                      |

**Your grade:** Participation & collaboration 25% · Problem sets 20% · Final project 20% · Midterm 10% · Quizzes 10% · Final exam 10% · Labs 5%

## Five things to do in week one

- 1 Log into [Canvas \(clarkcollege.instructure.com\)](https://clarkcollege.instructure.com) and read the announcements at the top.

- ② Complete the **first-week assignments** in two Canvas modules, **First Things First** and **Setting Up Your Coding Environment**. Each has its own due date; **the last is Thursday, September 24, at 11:59 PM Pacific**. Under Clark's first-week policy, anyone who hasn't finished them all will be dropped from the course on Friday, September 25.
- ③ Join the class **Slack**. No invite? Message me.
- ④ **Open cs50.dev** and sign in with your GitHub account. That's where you'll write your code.
- ⑤ Start **Module 1**: Lab 0, Problem Set 0, Collaborative Assignment 0.

#### WEEK ONE IS THE HEAVIEST WEEK – ON PURPOSE

Nearly everything on that list is a **one-time setup task**: make an account, install a tool, sign in once. You do it now so you never have to do it again. Getting your environment working *before* the real programming starts is what lets every week after this one be about Python instead of about tools — from Week 2 on, when something breaks, it's your code, not your setup.

The Thursday deadline isn't padding either: Clark's first-week policy sets it, so this work can't be spread out. After week one the course settles into a steady weekly rhythm, and it stays there.

**If any of it stalls, message me on Slack before Thursday rather than sitting on it.** Setup problems are completely routine, and they take minutes to fix together instead of hours alone.

---

## The course at a glance

*The practical details of this section: what it is, when it meets, and where.*

|                             |                                                                                                                                            |
|-----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Course</b>               | CTEC 121, Introduction to Programming and Problem Solving · Item 36001                                                                     |
| <b>When and where</b>       | Mondays and Wednesdays, 10:00 AM – 12:20 PM · Scarpelli Hall, Room 125                                                                     |
| <b>Quarter dates</b>        | Monday, September 21 – Friday, December 4, 2026 · Finals week December 7–10                                                                |
| <b>Sessions we lose</b>     | Wednesday, November 11 (Veterans Day) and Wednesday, November 25 (Thanksgiving).<br><b>Weeks 8 and 10 have one session instead of two.</b> |
| <b>Last day to withdraw</b> | Thursday, December 3, 2026                                                                                                                 |
| <b>Credits &amp; time</b>   | 5 credits · about 10 hours a week total — 4 hours 40 minutes in class, roughly 5–6 hours on your own                                       |
| <b>Modality</b>             | In person                                                                                                                                  |
| <b>Course site</b>          | <u>Canvas (<a href="https://clarkcollege.instructure.com">clarkcollege.instructure.com</a>)</u>                                            |
| <b>Materials cost</b>       | \$0                                                                                                                                        |
| <b>Prerequisites</b>        | Eligibility for ENGL& 101 or PTWR 135, and a “C” or better in PTCS 110 or a math course with a prerequisite of MATH 30 or higher           |

**What the catalog says:** fundamental concepts related to designing and writing computer programs and procedures. Topics include problem-solving techniques, program design, coding, debugging, testing, and documentation. The course stresses concepts common to many programming languages and introduces Python. It meets general education requirements for transfer degrees.

**What that means in practice:** you will learn to take a problem you don't yet know how to solve, break it apart, and write Python that solves it. You'll do that through labs, problem sets, quizzes, group work, and a final project of your own design.

**About CS50P:** this course is aligned with CS50 at Harvard University's free [CS50P "Introduction to Programming with Python"](https://cs50.harvard.edu/python/) ([cs50.harvard.edu/python](https://cs50.harvard.edu/python/)), so you can work through that course alongside this one if you want to. That's entirely optional — it isn't required, and it isn't part of your grade.

## Meet your instructor

*How to contact me, including student hours and how quickly to expect a reply.*

|                             |                                                                                                                                       |
|-----------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| <b>Name</b>                 | Bruce Elgort — please just call me Bruce                                                                                              |
| <b>Email</b>                | <a href="mailto:belgort@clark.edu">belgort@clark.edu</a>                                                                              |
| <b>Cell</b>                 | <a href="tel:(360)992-2951">(360) 992-2951</a> — call or text                                                                         |
| <b>Office</b>               | Scarpelli Hall Room 127 and Virtual (Zoom), plus before and after every class                                                         |
| <b>Student hours</b>        | On Zoom — days, times, and link are in Canvas. If none of those times work for you, message me on Slack and we'll find one that does. |
| <b>Best way to reach me</b> | <b>Slack.</b> Email works too; phone and Canvas messages reach me as well.                                                            |
| <b>Response time</b>        | Within 24 hours Monday-Friday; by Monday evening for weekend messages.                                                                |

I've taught web development and programming at Clark for 14 years, and I still get a kick out of watching someone's first working program run. Outside of class I tinker with new and emerging technology, usually with an extra-large cup of coffee nearby. In industry I'm known for co-developing Elguji's IdeaJam software.

**Coffee's on me.** My office, Scarpelli Hall 127, connects to our classroom, SHL 125, and there's a Keurig coffee maker in it, along with mugs, cream, and sugar. You're welcome to make yourself a cup — no need to bring your own mug.

I earned a Bachelor of Engineering in Electrical Engineering from the Stevens Institute of Technology in 1985, and a Master's in Engineering Management from NYU's Tandon School of Engineering in 1990.

### IF I GO QUIET

If a full day goes by and you haven't heard back from me, message me again. Something went astray. You are not bothering me.

## What I believe about teaching

*"Everyone you meet is fighting a battle you know nothing about. Be kind always."*

- Learning is slow. It takes time and repetition, and that is normal — not a sign you're bad at this.
- You'll gain far more from correcting your own answer than from me correcting it for you.

- **I'm not an answer key.** If I just told you “yes, that’s right,” you’d never build the ability to judge your own work — and there are no answer keys in a real job.
- Knowing *why* your answer is right matters as much as the answer.
- Asking questions is the single best thing you can do for yourself and your group.
- Every activity I assign, I’ve worked through myself, more than once, before you see it.

## What this class is like

### Why Python

Python is the friendliest first language there is. The syntax stays out of your way, it pushes you toward writing clean code, and it’s genuinely useful the moment you learn it — websites, data analysis, automation, machine learning. It’s also one of the most in-demand languages in the job market, which doesn’t hurt.

And no, it isn’t named after the snake. Guido van Rossum named it after the comedy show *Monty Python’s Flying Circus*, and the jokes are still hiding in the official documentation: where other languages use `foo` and `bar` as placeholder names, Python’s examples use `spam` and `eggs`.

Seeing square brackets and words like `function`, `range`, or `if` on your screen can be intimidating at first. That fades faster than you’d think. Coding is really just solving problems in small, careful steps, and that’s a skill you already have.

### How we spend our time

We’ll use lectures, videos, group activities, hands-on labs, peer critiques, and discussion. You’re expected to take part in class discussions and exercises, and I’d love it if you brought your own real-world examples into them.

Each week you’ll work through a Canvas module — Module 1 spans the first two weeks — with readings, videos, labs, a problem set, and a collaborative assignment. Most weeks also open with a short in-class quiz on the module you just finished — eight across the quarter, on the dates in the schedule. Modules open on Sundays, except Modules 0 and 1, which open on the first day of class.

**Want to work ahead?** Message me on Slack and I’ll unlock material early. This is especially worth doing if you’re also working through CS50P and wanting to earn the CS50 certificate.

#### COMPLETELY NORMAL IN HERE

- Your code breaking. Mine breaks too, usually live, in front of all of you.
- Looking up syntax you’ve already used fifty times. Professional programmers do it every day.
- Getting an error message longer than your program.
- Staring at a bug for twenty minutes and finding a missing colon.
- Asking a question you’re sure is obvious. It isn’t, and three other people were wondering the same thing.

### Attendance

A meaningful portion of your participation grade is earned in the room, so please plan to attend both sessions each week.

- **If you have to miss class**, let me know on Slack when you can. You’re still responsible for that week’s module and its deadlines.

- Missing class doesn't automatically cost you points. In-class exercises and the quizzes are the exceptions: for exercises there's a make-up you can do on your own (see [How participation works](#)), and one missed quiz is covered by the dropped-score policy.
- **If your attendance starts slipping for any reason** — illness, work, caregiving, anything — tell me early. Almost every problem is solvable in Week 3 and much harder to solve in Week 10.
- Absences for reasons of faith or conscience are accommodated. The college asks you to request this within the first two weeks of the quarter — details are in [Syllabus Information 26-27](#) ([clarkcollege.instructure.com/courses/2696810](http://clarkcollege.instructure.com/courses/2696810)) — so just let me know early.

## If you need an accommodation

I want this course to work for you. Start with Clark's [steps for requesting accommodations](#) ([www.clark.edu/campus-life/...](http://www.clark.edu/campus-life/)) through the Disability Access Center, then talk to me about what you need in this class specifically. DAC contact details and testing procedures are in the college's [Syllabus Information 26-27](#) ([clarkcollege.instructure.com/courses/2696810](http://clarkcollege.instructure.com/courses/2696810)) in Canvas. Once your accommodations are set, contact me so we can talk about this course specifically. If you have emergency medical information I should know, tell me by the end of week one.

## What you'll need

**Cost: \$0.** No textbook, no lab fees. All course materials are free, including resources aligned with CS50 at Harvard University.

*Everything you need for this course and what each item is for.*

| WHAT                            | NOTES                                                                                                                                                |
|---------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>A computer</b>               | Windows, macOS, or Linux — and a Chromebook works too, because you'll code in the browser.                                                           |
| <b><a href="#">cs50.dev</a></b> | Our coding environment. It runs in your browser, requires no installation, and comes with AI autocomplete already switched off. Sign in with GitHub. |
| <b>Internet access</b>          | Plus a backup plan for when your computer or connection fails. Clark's computer labs are listed in Module 0.                                         |
| <b>A web browser</b>            | Chrome, Firefox, Edge, or Safari.                                                                                                                    |
| <b>Slack</b>                    | Our class communication hub. Setup instructions in Module 0.                                                                                         |
| <b>A GitHub account</b>         | Free — it's how you sign into cs50.dev. Setup is in the Setting Up Your Coding Environment module; an existing account is fine.                      |
| <b>Free software</b>            | Anything else you need is free, and Canvas tells you exactly how to set it up.                                                                       |
| <b>The CS50 Duck</b>            | Your approved AI helper. Built into cs50.dev — nothing to install or sign up for.                                                                    |

You'll also need to be comfortable using Canvas ([tutorials here](#) ([www.clark.edu/academics/eLearning](http://www.clark.edu/academics/eLearning))) and searching online for answers. Canvas works on phones for reading, but you'll need a real computer to write and submit code.

## A note on the three outside accounts

Slack, GitHub, and cs50.dev are run by companies other than Clark, so a few things are worth knowing:

- You may use a display name instead of your full legal name. Just tell me which account is yours.

- Don't post personal or sensitive information — yours or anyone else's — in Slack or in a public GitHub repository.
- Each service sets its own privacy and accessibility terms, which Clark doesn't control.
- **Would rather not create one of these accounts?** Tell me in week one. There's an alternative for each, and using it won't affect your grade.

For Microsoft and Canvas accessibility and privacy details, see [Clark's eLearning policies page](http://www.clark.edu/academics/...) ([www.clark.edu/academics/...](http://www.clark.edu/academics/...)). As a Clark student you can also [download Microsoft Office for free](http://www.clark.edu/its/...) ([www.clark.edu/its/...](http://www.clark.edu/its/...)) — not required here, but useful elsewhere.

## How you'll be graded

*How your final grade is divided, with the weight of each category and what it covers.*

| CATEGORY                                             | WEIGHT | WHAT IT IS                                                                                         |
|------------------------------------------------------|--------|----------------------------------------------------------------------------------------------------|
| <b>Discussion, participation &amp; collaboration</b> | 25%    | Weekly reflections, in-class exercises, and nine collaborative assignments                         |
| <b>Problem sets</b>                                  | 20%    | One per module, due Sunday                                                                         |
| <b>Final project</b>                                 | 20%    | Assigned Week 5, due Friday, December 4 — firm, no three-day window — presented during finals week |
| <b>Midterm</b>                                       | 10%    | In class, Wednesday, October 28 — the Week 6 checkpoint, on paper                                  |
| <b>Quizzes</b>                                       | 10%    | Eight across the quarter, in class on paper — lowest score dropped                                 |
| <b>Final exam</b>                                    | 10%    | In class, Monday, November 30 — cumulative, on paper                                               |
| <b>Labs</b>                                          | 5%     | One or more per module, due Sunday                                                                 |

### THE DEADLINE RULE, ONCE

Labs, problem sets, and collaborative assignments are due **Sunday at 11:59 PM Pacific** and stays open for **three more days — through Wednesday at 11:59 PM — at full credit**. Three things sit outside it: the first-week assignments (all due by Thursday, September 24), anything taken in class, and the final project. **Canvas carries the exact date for every assignment; where it differs from this syllabus, Canvas is correct.**

### The details

- **Problem sets** — one per module. Close three days after the due date.
- **Labs** — one or more per module. Close three days after the due date.

- **Quizzes — eight across the quarter, taken in class on paper.** About ten minutes at the start of class, closed-book. They are short on purpose: the point is a regular, honest check that the code you turned in is code you can actually write.
  - **Your lowest quiz score is dropped**, and that drop *is* the make-up policy — it covers one missed quiz, not several.
  - **When:** each quiz is given at the start of the **first class session after that module's work is due**. In practice that means Mondays. They're marked on the schedule below, and I'll remind you the session before. You will never walk into a surprise quiz.
  - **The three-day window doesn't move the quiz.** Using those extra days on an assignment is fine — the quiz still happens on its date. So if you're going to run late on something, run late on the assignment, not on the studying.
  - **What's on it:** the module whose work was due that Sunday — material you have already worked through. Never something we haven't covered yet.
  - **Module 9 (regular expressions) has no quiz.** It's due the night before the final exam, so regular expressions are assessed on the exam instead.
- **Midterm** — in class on **Wednesday, October 28** (Week 6). About 50 minutes, on paper, closed-book, covering Weeks 1-5: functions, variables, conditionals, loops, and exception handling. Same format as the final exam, so it doubles as your dress rehearsal for it.
  - It lands five weeks before the **December 3** withdrawal deadline on purpose. You get a real, graded signal about where you stand while you still have room to act on it.
  - Like the final exam, it has **no three-day window**. If you cannot be there on October 28, talk to me *before* that date — or as soon as you can, if an emergency makes that impossible.
- **Final project** — assigned in Week 5, due **Friday, December 4 at 11:59 PM**, and presented to the class during **finals week (December 7-10)**. You submit first and present after, so you're never building and presenting in the same week. It includes a short written **ethics and limitations** piece: what ethical questions your program raises, and what it can't — or shouldn't — do. That part is graded, not optional.
  - **December 4 is firm — like the first-week assignments, it has no three-day window.** Presentations begin December 7, so I need every project in hand before then. If something is going to stop you, talk to me before December 4, or as soon as you can if an emergency makes that impossible.
- **Final exam** — in class on **Monday, November 30**. It's cumulative, on paper, and closed-book — with one exception: a short regular-expression syntax reference comes with those questions. Expect tracing code, predicting output, and writing short functions by hand. You've been doing all three since Week 1.
  - **Like the quizzes and the midterm, it has no three-day window.** If something will keep you from being there on November 30, contact me *before* that date — or as soon as you can, if an emergency makes that impossible — and we'll arrange something.
  - **Testing accommodations through DAC?** DAC needs at least five business days' notice for each exam, so set up both the midterm (October 28) and the final (November 30) in the first two weeks. November 30 is the tight one: Thanksgiving break eats most of that week, so book it by **Friday, November 20** at the latest. For the short weekly quizzes, tell me in the first two weeks — or as soon as the need comes up — and we will arrange your accommodation in class.
  - I'll post a review guide and a practice exam in Canvas by Week 9, so you can prepare before the break rather than during it. The guide says exactly which regular-expression patterns the exam can ask about; the syntax reference comes with the exam itself, so what you carry in is the thinking, not the punctuation.

Rubrics for every assessment live in the relevant Canvas module.

## How participation works

This is your biggest category, so here is exactly how you earn it.

*The three parts of the participation grade and what each is worth.*

| COMPONENT                                                       | SHARE OF YOUR FINAL GRADE |
|-----------------------------------------------------------------|---------------------------|
| In-class participation and group work                           | 12%                       |
| Collaborative assignments (nine across the quarter)             | 8%                        |
| Weekly written/video reflections and discussion posts in Canvas | 5%                        |

- **Graded on effort, not correctness.** Show your reasoning and you get full credit even when the answer turns out to be wrong. A blank or one-word submission doesn't.
- Reflections and collaborative assignments can be done outside class, so missing a session doesn't cost you those points.
- **In-class participation** is the one part that needs you in the room. Miss a class? Message me on Slack and I'll give you an equivalent to do on your own.
- **If speaking up in class or working in groups is hard for you** — including as part of a DAC accommodation — tell me in the first two weeks, or as soon as the need comes up. There's a written alternative for every in-class component. Setting it up is routine and completely unremarkable.

## Grading scale

*Percentage ranges and the letter grade each one earns.*

| PERCENT | GRADE | PERCENT   | GRADE |
|---------|-------|-----------|-------|
| 94–100% | A     | 80–83%    | B-    |
| 90–93%  | A-    | 77–79%    | C+    |
| 87–89%  | B+    | 70–76%    | C     |
| 84–86%  | B     | 60–69%    | D     |
|         |       | Below 60% | F     |

- **A “C” or better** is required for this course to count toward a degree or certificate.
- **Rounding:** final percentages round to the nearest whole number, so 89.5% becomes 90% and earns an A-. I don't round past that. What I *will* do is talk with you early about how to move your grade — please don't wait until Week 11 to ask.
- **Incompletes:** an “I” is possible only if you've completed most of the course in good standing and something unexpected stops you from finishing. It takes a written agreement between us with a deadline, and it isn't a substitute for withdrawing.

## Late work

**One rule: work is due Sunday at 11:59 PM and stays open three more days at full credit.**

- No penalty, no permission needed, no explanation required inside that window. It's built into every regular take-home deadline.
- Canvas will stamp those submissions “late.” **Ignore the label** — it doesn't affect your grade.

- **Three things sit outside the window.** First, the **first-week assignments** in First Things First and Setting Up Your Coding Environment, all due by **Thursday, September 24, at 11:59 PM Pacific**: under Clark’s first-week policy, I drop anyone who hasn’t finished them on Friday, September 25. If something is in your way, message me on Slack before Thursday. Second, **anything taken in class** — the weekly quizzes, the midterm (October 28), and the final exam (November 30) all happen in the room. For quizzes, the dropped lowest score covers a miss; for the midterm or the final, talk to me *before* the date. Third, the **final project**: December 4 is firm, because presentations begin December 7.
- **When something bigger is going on** — illness, work, family, a genuinely brutal week — contact me *before* the deadline and we’ll set a new date. I grant these. I just need to hear from you — and if an emergency makes reaching me beforehand impossible, tell me as soon as you are able.

Work that lands weeks or months late has lost most of its value, because it no longer connects to anything the class is doing. That’s the reason for the window, and it’s why talking to me early works so much better than going quiet.

## Feedback

I aim to return feedback within a week of the due date, while the material is still fresh. If you want extra help, reach out and we’ll find a time.

## Weekly schedule

Subject to change — I’ll announce any updates in class, on Canvas, and on Slack.

*The quarter week by week: dates, topics, quiz and exam dates, and what is due each week.*

| WEEK | DATES           | TOPIC                                                                              | WHAT’S DUE                                                                                                     |
|------|-----------------|------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|
| 1–2  | Sept 21 – Oct 2 | Course intro · Setting up your environment · Functions and variables               | <b>Thursday, September 24</b><br>First-week assignments · no window<br>Lab 0 · Problem Set 0 · Collaborative 0 |
| 3    | Oct 5 – 9       | Conditionals · <b>Quiz 0 in class Monday</b>                                       | Lab 1 · Problem Set 1 · Collaborative 1                                                                        |
| 4    | Oct 12 – 16     | Loops · <b>Quiz 1 in class Monday</b>                                              | Lab 2 · Problem Set 2 · Collaborative 2                                                                        |
| 5    | Oct 19 – 23     | Exception handling · <b>Quiz 2 in class Monday</b> · <b>Final project assigned</b> | Lab 3 · Problem Set 3 · Collaborative 3                                                                        |
| 6    | Oct 26 – 30     | Libraries · <b>Quiz 3 in class Monday</b> · <b>Midterm in class Wed Oct 28</b>     | Lab 4 · Problem Set 4 · Collaborative 4                                                                        |
| 7    | Nov 2 – 6       | Unit tests · <b>Quiz 4 in class Monday</b>                                         | Lab 5 · Problem Set 5 · Collaborative 5                                                                        |
| 8    | Nov 9 – 13      | File input/output · <b>Quiz 5 in class Monday</b> · <i>no class Wed Nov 11</i>     | Lab 6 · Problem Set 6 · Collaborative 6                                                                        |
| 9    | Nov 16 – 20     | Object-oriented programming · <b>Quiz 6 in class Monday</b>                        | Lab 7 · Problem Set 7 · Collaborative 7                                                                        |
| 10   | Nov 23 – 27     | Regular expressions · <b>Quiz 7 in class Monday</b> · <i>no class Wed Nov 25</i>   | Lab 8 · Problem Set 8 · Collaborative 8                                                                        |

| WEEK          | DATES          | TOPIC                                           | WHAT'S DUE                                                             |
|---------------|----------------|-------------------------------------------------|------------------------------------------------------------------------|
| 11            | Nov 30 – Dec 4 | Final exam Mon Nov 30 · <i>project work Wed</i> | <b>Friday, December 4</b><br>Final project. Firm: no three-day window. |
| <b>Finals</b> | Dec 7 – 10     | Final project presentations                     | Present your project — schedule posted in Canvas                       |

**Exact due dates live in Canvas.** Every assignment carries its own due date there, and Canvas is the one that counts — if a date in this syllabus ever disagrees with Canvas, go with Canvas. The dates printed here are the fixed ones, and they will not move: the first-week deadline, the midterm, the final exam, the final project, and the last day to withdraw.

**The weekly rhythm:** labs, problem sets, and collaborative assignments are due **Sunday at 11:59 PM Pacific** and stay open three more days at full credit, through the following **Wednesday at 11:59 PM**. Quizzes, the midterm and the final exam are taken in class, so they aren't part of that window — and neither are the first-week assignments or the final project.

From Week 5 on, you're also working on your final project alongside each week's module.

#### PLAN AROUND THE END OF THE QUARTER

Week 10 has only one class session, it's the week of Thanksgiving, and the final exam is the Monday right after the break. Regular expressions land in that week on purpose — they're well suited to focused practice with the documentation open, which is how you'll use them in real work. Module 9's work is due the **Sunday night before the exam**. Finish it *before* the weekend if you possibly can. The three-day window does technically carry it into the following week, but don't plan on that: it would land two days before your final project is due. Treat the window as a safety net here, not a schedule. Do your exam reviewing during Week 9 and the first half of Week 10, when the review guide is already posted, rather than over the holiday. Week 8 also has a single session.

## Course agreements

### Communication

**Slack is the fastest way to reach me**, and it's where you can ask questions, help each other, and share things you find. Everyone gets an invitation; if yours didn't arrive, contact me. Check Canvas announcements whenever you log in — they're at the top of the page.

On netiquette: there are real people on the other side of every message. Use the same courtesy online that you'd use face to face.

**Bug of the week.** Post the strangest bug you fixed in the **#bug-of-the-week** Slack channel: what broke, how long it took to find, and what it turned out to be. It isn't graded. It's purely for bragging rights, and the sillier the cause, the better.

### AI guidelines

AI is part of this course. It's also the easiest way to accidentally skip the learning you're paying for. So this policy is about what you must be able to **do**, not about a list of banned brand names.

**The standard:** everything you submit must be code whose decisions you can explain, and that you can change on your own, without AI help. If you couldn't talk me through why it works and modify it when asked, it isn't yours to submit. The weekly quizzes, the midterm, the final exam, and your project presentation all check exactly that — which is why leaning on AI now mostly shows up as a bad grade later.

*What AI help is allowed on take-home work, what is not, and where you will write your code.*

|                                          |                                                                                                                                                                                                                                                                                                                                 |
|------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Always allowed</b>                    | The <b>CS50 Duck Debugger</b> , built into cs50.dev. It asks you questions instead of handing you answers. That's the point.                                                                                                                                                                                                    |
| <b>Allowed for learning, with a note</b> | Asking any AI to explain a concept, unpack an error message, or summarize documentation — the same way you'd use a search engine or a textbook. If it shaped what you turned in, say so in a code comment, the same as any other source. What stays off-limits is letting it write the code.                                    |
| <b>Not allowed on submitted work</b>     | Any other AI that suggests, completes, or generates code — ChatGPT, Claude, Gemini, GitHub Copilot, Cursor, and the AI features now baked into editors and browsers — unless that assignment says otherwise; see <b>When AI is assigned</b> below. Submitting AI-written code is treated like submitting another person's code. |
| <b>Where you'll write code</b>           | <b>cs50.dev</b> , CS50's browser-based environment. Copilot and inline AI autocomplete are already turned off there, which is one less thing for you to get wrong. If you'd rather work in your own editor, that's fine — but then it's on you to make sure no AI completion is running.                                        |
| <b>When AI is assigned</b>               | Some assignments will ask you to use AI beyond the Duck. Those will say so explicitly, name the allowed tools, and may ask you to turn in your prompts.                                                                                                                                                                         |

**Everything in this table is about work you do outside class.** The quizzes, the midterm and the final exam are closed-book and on paper — no AI, notes, or web, including the Duck.

**If I think AI wrote your code,** I'll ask you to walk me through it — what a section does, why you did it that way, how you'd change it. That's a conversation, and it happens before any finding is made. I don't use AI-detection software to assign grades; those detectors are unreliable and I won't grade you on one.

**Not sure whether something's allowed? Ask me on Slack before you submit.** Asking has never once counted against a student.

## Academic honesty

My approach here is best described as **reasonable**. Working with other people genuinely helps you learn. But there's a line between getting help and handing in someone else's work, and this is where it sits.

**The essence of everything you submit must be your own.** The general rule when asking for help: *you may show your work to others, but you may not look at theirs.*

**This table is about take-home work you turn in as your own** — labs, problem sets, and your final project. It does **not** describe the quizzes, the midterm or the final exam: those are closed-book and taken in class, so none of the help below is available during them. **The collaborative assignments are the deliberate exception:** on those, working together, reading each other's code, and turning in a joint product *is* the assignment. Each one tells you who you're working with and what you may share. If an assignment doesn't say it's collaborative, treat it as individual and use the table below.

*Where the line sits on take-home work: getting help on one side, handing in work that is not yours on the other.*

## THIS IS FINE

## THIS IS NOT

|                                                                                                                                                                                  |                                                                                                     |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| Talking a problem through with a classmate in words, and citing that conversation in a comment                                                                                   | Looking at a classmate's solution before you've submitted your own                                  |
| Showing your code to someone so they can help you find your bug                                                                                                                  | Sharing your solution with someone who's stuck on it                                                |
| Helping a classmate debug <i>after</i> you've submitted that part yourself                                                                                                       | Splitting an <b>individual</b> assignment with someone and combining the pieces                     |
| Whiteboarding an approach in diagrams or pseudocode                                                                                                                              | Whiteboarding in actual code                                                                        |
| Borrowing a few lines you found online and citing where they came from                                                                                                           | Searching for, or asking for, outright solutions to assigned work                                   |
| Using the web to learn more, look things up, or solve technical trouble                                                                                                          | Copying someone's solution and basing yours on it                                                   |
| Working with a tutor who helps you learn                                                                                                                                         | Paying anyone for work you submit                                                                   |
| Using the CS50 Duck · asking another AI to explain a concept or an error message, and citing it in a comment · using AI beyond the Duck when an assignment explicitly assigns it | Using any other AI to write, complete, or suggest code you submit, unless that assignment says to   |
| Reusing your own work from <i>this</i> course                                                                                                                                    | Reusing work from another course, or giving your solutions to anyone — this quarter or a future one |

**Also not okay:** using an account that isn't yours, manipulating scores, and finding a flaw in Canvas or an autograder that affects grades without telling me.

**Cite your sources.** If you bring in code or a technique from outside our lessons, say where it came from in a code comment. Same idea as citing a source in an essay: [Clark Libraries' plagiarism guide](https://clark.libguides.com/c.php) ([clark.libguides.com/c.php](https://clark.libguides.com/c.php)) is a good primer.

**If something crosses the line,** consequences follow [Clark's academic dishonesty policy](#): no credit on the assessment, a failing course grade for serious or repeated violations, and possible referral to Student Conduct. You have the right to respond before any finding is made and to appeal through the division chair.

## THE REGRET CLAUSE

Most academic dishonesty I have seen came from a deadline, a bad week, and a blank editor lining up badly — not from someone setting out to cheat. **The Regret Clause exists so one of those moments doesn't decide your quarter.** Tell me you crossed the line, and once per quarter it costs you nothing.

### How it works

- **You get one, for the whole quarter.** Not one per assignment. One.
- **Come to me before I have noticed, or after I have asked you about a piece of work.** Both count. If I ask you to walk me through your code and you would rather tell me the truth than perform the walk-through, say so right there — that is the moment this was built for.
- **You redo the work honestly, and I grade it normally.** No late penalty, no lowered ceiling, no note in the gradebook. That is what "no penalty" means here: your grade is whatever the honest work earns.
- **Nothing goes to Student Conduct,** and no finding is recorded against you. I keep a private note that you have used it, because there is only one.

- **Tell me in writing** — Slack or email — with the assignment name, what you did, and what you will do instead. Three sentences is plenty. “I used an AI to write the loops in Problem Set 3 and I will rewrite them myself” is a complete disclosure. You do not owe me an explanation of your life.

### When to use it, and when the redo is due

- **Use it as soon as you realize.** Waiting doesn’t improve anything, and a disclosure that arrives after I have already started a conversation with your division chair is too late to be this.
- **The last day to use it is Friday, December 4** — the final day of the quarter. After that, grades are being finalized and there is no room left to redo anything.
- **We set the redo date together, in writing, the same day you tell me.** Expect **about a week** for a lab or a problem set. Later in the quarter it will be shorter — a disclosure about Module 9 or the final project may get only a few days, because the work still has to be graded before the quarter closes.
- **If the redo date passes and nothing arrives,** the clause has been used and the original consequences apply. Tell me if you need the date moved; that conversation is always available.

*What the Regret Clause covers and what it does not.*

| THE CLAUSE COVERS                                                                 | THE CLAUSE DOES NOT COVER                                                                                            |
|-----------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| Take-home work — labs, problem sets, collaborative assignments, the final project | The quizzes, the midterm, and the final exam. Those are proctored and closed-book, and go through the normal process |
| An AI writing, completing, or suggesting code you submitted                       | A second violation. There is one clause; after it, the consequences above apply                                      |
| Work you copied from a classmate, from the web, or from a previous quarter        | Using an account that isn’t yours, altering scores, or exploiting a grading flaw                                     |
| Collaboration that went further than an individual assignment allowed             | Someone else’s violation. It is yours to use, not to lend                                                            |

**What it can’t do.** It can’t give you back the practice you skipped. The quizzes, the midterm and the final exam are on paper with nothing to lean on, and your project presentation is a conversation about code you are expected to know. Work that isn’t yours doesn’t fail you in Week 4 — it fails you in Week 11, quietly, when the material isn’t there. So use the clause, then use the reset: come to student hours, ask on Slack, work the problem with the Duck until you can explain it out loud. That part is what actually saves the quarter.

**When in doubt, ask me first — in writing, so we both have the answer.** Asking in good faith never counts against you.

### Classroom norms

- Complete each week’s module — readings, assignments, and discussions — by the due date.
- Take part in class discussions and exercises, and bring your own experiences into them.
- Be a courteous member of the room and don’t get in the way of anyone else’s learning.
- Academic and intellectual honesty are expected at all times.
- Tell me as early as you can about anything that affects your ability to take part fully.

### Recording class sessions

**You’re welcome to record class for your own study use** — no permission needed. Please don’t post or share recordings, screenshots, or classmates’ work outside this class. People share half-finished code and honest confusion in a programming class, and that only works if the room feels safe. If DAC

has approved recording as an accommodation, you never need to explain that to anyone. If I record a session, I'll say so and post it in Canvas.

## Changes to this syllabus

This is a plan, not a contract, and I may adjust it during the quarter. Anything that affects your grade or your deadlines gets announced in Canvas and on Slack, and I'll post an updated syllabus.

## Withdrawing from the course

The last day to withdraw is **Thursday, December 3, 2026**. If you stop attending without formally withdrawing, you'll receive an "F." Deadlines are on Clark's [Important Dates & Deadlines](http://www.clark.edu/about/...) ([www.clark.edu/about/...](http://www.clark.edu/about/...)) page.

**Before you withdraw, please talk to me.** There's often an option you haven't considered.

## What you'll be able to do by the end

- **C1.** Show how computer operations and algorithmic thinking are applied to solve problems.
- **C2.** Demonstrate working knowledge of fundamental computer concepts and iteration.
- **C3.** Identify problems, solve them algorithmically, and write programs that address them.
- **C4.** Discuss ethics in computing and the limitations of what computing can do.

*The four course outcomes, where each is taught, and how each is graded.*

| OUTCOME   | TAUGHT IN                                            | GRADED THROUGH                                                                   |
|-----------|------------------------------------------------------|----------------------------------------------------------------------------------|
| <b>C1</b> | Weeks 1-4: functions, variables, conditionals, loops | Labs 0-2 · Problem Sets 0-2 · Quizzes 0-2 · Midterm · Final exam · Final project |
| <b>C2</b> | Weeks 3-7: loops, exceptions, libraries, unit tests  | Labs 1-5 · Problem Sets 1-5 · Quizzes 1-5 · Midterm · Final exam                 |
| <b>C3</b> | Every week, most heavily Weeks 5-10                  | Problem sets · Collaborative assignments · Final exam · Final project            |
| <b>C4</b> | Raised in Week 1, revisited as topics come up        | The final project's <b>ethics and limitations</b> component                      |

## College-wide abilities and program outcomes

**College-wide abilities.** Clark identifies six abilities at the foundation of its educational emphasis: communication, critical thinking and problem solving, information and technology, effective citizenship, life-long learning, and global/multicultural awareness. This course emphasizes **information technology, critical thinking and problem solving**, and **communication**.

**Program outcomes (Web Development AAT).** As an introductory programming course, CTEC 121 contributes directly to **P1** (web foundations), **P4** (web development), **P5** (professional practices), **P6** (written communication), **P7** (interpersonal skills), and **P8** (explaining a strategy for a quantitative problem). P2 (web media) and P3 (web design) are developed in other courses in the degree.

# College policies

## COLLEGE-WIDE SYLLABUS INFORMATION

Clark College now publishes the policies and support services that apply to every class in one place in Canvas: **Syllabus Information 26-27** ([clarkcollege.instructure.com/courses/2696810](https://clarkcollege.instructure.com/courses/2696810)). It is part of this syllabus, so please read it. It covers disability accommodations (DAC), non-discrimination, Title IX, student rights and responsibilities, the Code of Student Conduct, absences for reasons of faith or conscience, emergency preparedness and safety, and campus support such as tutoring, the Tech Hub, advising, and the Basic Needs Hub.

Two college topics aren't on that page, so they stay here.

## Academic dishonesty

Academic dishonesty usually happens when students are stressed and out of time, not because they set out to cheat. Two main forms:

- **Plagiarism** — not crediting or properly citing sources for your work.
- **Cheating** — giving or getting help that isn't allowed on an assignment.

**How to stay clear of it:** ask me when you're unsure, cite your sources, turn in original work, don't submit the same work in two classes without permission, and never fabricate results or citations.

**Consequences:** a failing grade on the assignment or, in extreme cases, the course. Students may also be referred to the Student Conduct Office, which can require learning activities or, in extreme or repeated cases, dismissal from a program. To challenge a finding, you can file a grade appeal starting with the division chair.

## Emergencies

*What to do in an emergency or a campus closure.*

| SITUATION                           | WHAT TO DO                                                                                                                                                                  |
|-------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Inclement weather or closure</b> | Check <a href="http://www.clark.edu">www.clark.edu</a> or call 360-992-2000 first — those are the official sources.                                                         |
| <b>Emergency alerts</b>             | Register your phone and email at <a href="http://flashalert.net">flashalert.net</a> ( <a href="http://www.flashalert.net">www.flashalert.net</a> ). Do this now, not later. |
| <b>Fire alarm</b>                   | Leave by the nearest exit (maps are in the hallways), take belongings only if it's safe, stay at least 50 feet from the building, and don't re-enter until told.            |

Parking lots are identified by color and number — use those to tell security or emergency responders where you are.

## Where to get help

**Asking for help early is the single strongest predictor of finishing this course well.** Please use these.

## COURSE HELP

*Where to go for help with the course material and your code.*

IF YOU NEED

GO TO

**IF YOU NEED****GO TO**

Help with the material or an assignment

Message me on Slack, or come to student hours

A second set of eyes on your code

The CS50 Duck in cs50.dev, or ask in Slack

**About that duck.** Programmers have a trick called *rubber duck debugging*: explain your code, line by line, out loud, to a rubber duck, and somewhere around line six you'll hear yourself say what's wrong. That's where the CS50 Duck gets its name — and it's why you'll get a real rubber duck of your own on the first day of class. Keep it next to your keyboard. It's a very good listener.

**TECH HELP**

*Where to find a computer or set up your Clark accounts.*

**IF YOU NEED****GO TO**

A computer to work on

Open computer labs ([www.clark.edu/its/...](http://www.clark.edu/its/...)), including weekends

To borrow a laptop

Laptop request form ([www.clark.edu/its/...](http://www.clark.edu/its/...))

Account setup

All four Clark student accounts ([www.clark.edu/its/...](http://www.clark.edu/its/...)) · student email ([www.clark.edu/its/...](http://www.clark.edu/its/...)) · ctcLink ([www.clark.edu/its/...](http://www.clark.edu/its/...))**CAMPUS AND LIFE HELP**

Tutoring, librarians, the Tech Hub and Canvas help, Student Success Coaches, the Student Wellness Center, the Basic Needs Hub and Penguin Pantry, advising, career services, and the Veterans Center are all listed, with contact details, in the college's Syllabus Information 26–27 ([clarkcollege.instructure.com/courses/2696810](http://clarkcollege.instructure.com/courses/2696810)) in Canvas.

**IF MONEY IS TIGHT**

If you're having trouble affording food or don't have stable housing, please contact the Basic Needs Hub — and if you're comfortable telling me, do. I may know of resources you haven't found yet, and it stays between us.

Programming is learned by doing it badly, repeatedly, until it clicks.

Everyone's code breaks. Everyone gets stuck. The students who do well in here aren't the ones who never get stuck — they're the ones who ask sooner.

So ask. I'm on Slack, and I'm glad you're here.

— Bruce